

**backendのAPI動作確認**

# curlコマンドによる登録テスト

フロントエンドがない状態でも、バックエンドの「窓口（API）」が正しく機能しているか確認するのは、非常に重要なステップです。

GitHub Codespacesの**新しいターミナル**（Spring Bootを動かしているのとは別のタブ）を開いて、以下の手順でテスト用データを送ってみましょう。

以下のコマンドをコピーして、ターミナルに貼り付けて実行してください。

```
curl -X POST -H "Content-Type: application/json" \
-d '{
  "userId": "test-user@example.com",
  "userName": "テストユーザー",
  "password": "mysecretpassword",
  "about": "これはテスト登録です。"
}' \
http://localhost:8080/api/users/register
```

## コマンドの解説

- `-X POST` : データを送信する（POSTメソッド）ことを指定。
- `-H "Content-Type: application/json"` : 送るデータをJSON形式に指定。
- `-d '{...}'` : 実際に送るデータの中身。
- `http://localhost:8080/api/users/register` : `UserAccountController` で作成した受付窓口の住所。

## 成功かどうかの判定

実行後、ターミナルに以下のようなレスポンスが返ってくれば成功です！

```
{
  "accountId": "a1b2c3d4-e5f6-...", // UUIDが自動生成されている
  "userName": "テストユーザー",
  "userId": "test-user@example.com",
  "password": "$2a$10$...", // パスワードがハッシュ化（暗号化）されている！
  "isAuth": 0,
  "quitDemand": 0,
  "about": "これはテスト登録です。",
  "createdAt": "2026-05-07T..." // 日時が自動セットされている
}
```

## チェックポイント

1. password: 入力した `mysecretpassword` そのものではなく、`$2a$10$...` という長い文字列に化けていますか？これが **BCryptによるハッシュ化** が成功している証拠です。
2. accountId: 何も指定していませんが、ランダムな英数字（UUID）が入っていますか？これが **Entityでの自動生成** が機能している証拠です。

## エラーが出た場合

- 400 Bad Request: 同じメールアドレスですでに登録されている可能性があります（Serviceで書いた重複チェックが効いています）。
- Connection refused: Spring Bootが未起動か、ポート番号が誤っています。

まずはこのコマンドを一度実行してみて、どのような結果が返ってくるかを確認してみてください。

## ターミナル上で何のレスポンスも返ってこなかった場合

何も表示されずに終了したというのは、一見不安になりますが、実は「通信自体は届いたが、レスポンスの受け取り方で何かが起きた」ということを示しています。

通常、`curl` でエラーが起きれば「Connection refused」などが出ます。

何も出ないのは、サーバー側が処理をしたものの、ターミナル側で表示しきれていないか、処理が静かに終わってしまった可能性があります。

より詳細な状況を把握するために、\*\*「詳細ログ付きのcurlコマンド」\*\*を試してみましょう。

## 詳細ログ付きで再送する

以下のコマンドをコピーして実行してください。末尾に `-v`（verbose：詳細）を追加しています。

```
curl -v -X POST -H "Content-Type: application/json" \
-d '{
  "userId": "test-user@example.com",
  "userName": "テストユーザー",
  "password": "mysecretpassword",
  "about": "これはテスト登録です。"
}' \
http://localhost:8080/api/users/register
```

## これで何がわかるか？

- 画面に `>` や `<` で始まる大量の文字が出ます。
  - `< HTTP/1.1 200 OK` と出ていれば、**実は登録に成功しています**。
  - `< HTTP/1.1 404` や `500` の場合、プログラム側のどこかに不備があります。

## Spring Boot側の「ログ」を確認する

`curl` を打った瞬間、Spring Bootを起動している方のターミナルに何か変化はありませんでしたか？

- \*\*白い文字（INFOログ）\*\*が新しく流れていれば、通信は届いています。
- \*\*赤い文字（ERRORログ）\*\*が出ていれば、データベースへの保存などで失敗しています。

## 考えられる「よくある原因」

もし上記でも何も出ない場合、以下の2点が怪しいです。

1. **ポートの不一致**: Spring Bootが実は `8080` 以外で立ち上がっていないか？（起動ログに `Tomcat initialized with port(s): 8080 (http)` と出ているか確認）。
2. **パッケージ名の不整合**: 各Javaファイル（Entity, Repository, Service, Controller）の一番上にある `package ...;` の記述が、実際のフォルダ構成と1文字でもズレていると、Springが「これはコントローラーだ！」と認識してくれず、無視（スルー）されてしまいます。

まずは `-v` 付きのコマンドを実行してみて、`< HTTP/1.1 ...` の後続く数字が何番になっているかを確認してみましょう。



## 401エラーの場合の対処法

curlコマンドを送信したことで返ってきたレスポンスに `HTTP/1.1 401` という数字が記載されていた場合、これは「Unauthorized（認証されていません）」ということを意味します。

Spring Bootのプロジェクト作成時に「Spring Security」というライブラリを入れたため、標準設定では**すべての通信にログイン（ユーザー名とパスワード）が必要なロックがかかった状態**になっています。

これが原因でこの401エラーが発生してしまったのです。

ターミナルでのレスポンスでは、`X-Frame-Options: DENY` なども表示されているはずなのですが、これはセキュリティのための壁がしっかり立っている証拠です。今はテストのために、この壁を一時的に開ける必要があります。

`application.properties` に以前「セキュリティを無効化する」設定を1行書きましたが、ライブラリのバージョンによってはそれだけでは不十分な場合があります。

最も確実なのは、「**セキュリティ設定用のクラス**」を作成して、`/api/users/register` へのアクセスを**全許可**する設定を書くことです。

## 1. 新しいクラスの作成

`backend/src/main/java/com/example/demo` の中に、新しく `config` というフォルダを作り、その中に `SecurityConfig.java` というファイルを作成してください。

## 2. コードの記述

以下の内容をコピー & ペーストしてください。

```
package com.example.demo.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.web.SecurityFilterChain;
```

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable()) // テスト用なのでCSRF保護を無効化
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/users/register").permitAll() // 登録窓口だけは誰でもOKにする
                .anyRequest().authenticated() // それ以外はロック
            );
        return http.build();
    }
}
```

### 3. 再起動と再テスト

1. `SecurityConfig.java` を保存します。
2. Spring Bootを `Ctrl + C` で一度止めてから、再度 `./mvnw spring-boot:run` で起動してください。
3. 起動後、先ほどの `curl -v ...` のコマンドをもう一度実行してみてください。

## 次の結果はどうなる？

今度は `< HTTP/1.1 200 OK` が返ってくるはずです！ `< HTTP/1.1 200` というステータスコードに加え、最後に返ってきた JSON データの内容に注目してください。

## ここが「成功」の証拠

- `"accountId": "04ed4539-..."` : Java側で指定しなかったIDが、UUIDとして自動生成されている。
- `"password": "$2a$10$..."` : `mysecretpassword` が、解読不能なハッシュ値に変換されている。
- `"createdAt": "2026-05-07T..."` : サーバーの時間で正しく登録日時が記録される。

これで、「**フロントエンド** → **コントローラー** → **サービス** → **リポジトリ** → **データベース**」というバックエンドの全行程が1本の線で繋がったのです。

# データベースの中身を自分の目で見てみよう

現在はメモリ上のデータベース（H2 Database）を使用しているため、ブラウザからその中身を直接のぞき見ることができます。

1. Spring Boot を動かしたまま、ブラウザで以下のURLを開きます。

`https://（あなたのCodespacesのURL）/h2-console`

（ポート8080の地球儀マークで開いたURLの末尾を `/h2-console` に書き換える）

2. ログイン画面が出たら、以下を確認して「**Connect**」を押します。

○ **JDBC URL:** `jdbc:h2:mem:testdb`

○ **User Name:** `sa`

○ **Password:** （空欄）

3. 左側のツリーに `APPLY_USER_ACCOUNTS` というテーブルがあるはずです。

4. SQLコマンド `SELECT * FROM APPLY_USER_ACCOUNTS;` を入力して「Run」を押してみてください。

先ほど `curl` で送ったデータが1行表示されれば、データの保存も完璧です！